

Session_08 activities worksheet

This Session is for learning purposes only and will not be collected. Thus, there is no due date. If you are behind on other Sessions, focus on those.

Online listings: `diffeq_oscillations_arg.cpp` and `diffeq_oscillations_xml.cpp` + `XmlParameters.h` + `XmlParameters.cpp` printouts.

This Session focuses on techniques rather than on the physics. It will show you different techniques to provide parameters to a program and to work with the output of a program. This Session is optional, so you do **not** have to upload your code and results. This will allow the entire class to catch up a bit.

Your goals for today (and ...):

- Understand input and output redirection and pipes
 - Modify a program to use standard error and standard output
 - Understand command line arguments
 - Use XML files to define parameters
-

A) Input redirection

We are going to work with the `diffeq_oscillations.cpp` code from Session_07, which is again included in your package for this Session. You will need to use `make_diffeq_oscillations` again to make it.

1. Open an editor for an empty file `oscillator_parameters.inp`. Compile the `diffeq_oscillations` as before and start it. Change a few parameters and as you are doing it simultaneously type the same answers into the `oscillator_parameters.inp` file in the editor each into its own line. Save the `oscillator_parameters.inp` file
2. Run the oscillator program using `./diffeq_oscillations.x < oscillator_parameters.inp`. *Does it work?*
→ Yes, it works.
3. *What do you find difficult about this approach of setting parameters?*
→ I was a little confused about what exactly I should put into the `.inp` files, but after I figured out I needed to input each number I input manually, it was simple.

B) Output redirection

If we are reading input for a file, the current output is not that useful. Thus, we are learning how we can redirect it.

1. Run `./diffeq_oscillations.x > diff_oscillations.out`. Afterwards check your directory with `ls`. Use `cat` to output the content of any file of interest (or alternatively

open it/them in an editor). *What happened?*

→ It outputs the output from the terminal into `diff_oscillations.out`, but hangs because no input is being provided.

2. Try `./diffeq_oscillations.x < oscillator_parameters.inp > /dev/null`. Check out the file `/dev/null` with `cat`.
3. Change the source code of `diffeq_oscillations.cpp` to first comment out all lines that involve `cout` and then remove the file opening and closing commands for the output file `out`. Then, replace `out` by `cout` in all the statements that produce the actual output. Run the program as `./diffeq_oscillations.x < oscillator_parameters.inp > diff_oscillations.out` and look at the result.
4. Go back to the statement in the source code that outputs the initial energies (you should have commented it out in the previous step). Remove the comments and change `cout` to `cerr`. Rerun your program. *What happens now?*
→ The output file still only has the `cout` statements, but the `cerr` line outputs to the terminal.
5. Now run `./diffeq_oscillations.x < oscillator_parameters.inp | wc -l` and
`./diffeq_oscillations.x < oscillator_parameters.inp | grep -v # | wc -l` *What do the outputs mean?* [Hint: if you do not know what a command line switch such as `-f` on `grep` means, just say `man grep`.]
→ The pipe `wc -l` outputs the word count of the output, specifically the number of lines outputted. The `grep` line hides the lines that start with a comment (`#`) and then counts the number of lines from there.

C) Command line arguments

Another way to provide parameters is command line arguments. The program `diffeq_oscillations_arg.cpp` with corresponding makefile `make_diffeq_oscillations_arg` represents a minimally modified version of `diffeq_oscillations.cpp` to illustrate how this works. It reads the values that the original version would have expected to be typed at the console in the same order from the command line. I am not claiming that this is the most elegant way for using command line arguments.

1. Look at all places where `argc` and `argv` appear in the `diffeq_oscillations_arg.cpp`. *List any that you do not understand.*
→ I understand how `argc` and `argv` work, and I understand how they are being used.
2. Compile the new program. *What did you have to add to make it work?*

→ It compiles and runs but does nothing without providing command-line arguments. To run as-

is, you have to do `./diffeq_oscillations_arg.x 0 0`, which runs the default setup.

3. Run the program using `./diffeq_oscillations_arg.x 3 3 0 0`. *What did it do?*

→ It runs the program with $p=3$.

4. *What would you change in the command line argument syntax that this program expects to make this more useful (you do not have to know how you would actually implement this)?* [Hint: Think about how unix commands typically set parameters.]

→ It would be better to have flags so you can set different parameters easier. For example it would be nice to do `-p 3` to set $p=3$, and then use `-m 2` to use $m=2$, and so on.

D) Extensible Markup Language

One of the (in my opinion) most versatile ways of providing parameters to a program is by using a parameter file in the extensible markup language (XML). The program `diffeq_oscillations_xml.cpp` with makefile `make_diffeq_oscillations_xml` provides an example.

1. Look through the makefile. *List all the things that were added in order to use the libxml2 library.*

→ We include all the XML C++ files, like `XmlParameters.h`, we add LDFLAGs like `-lxml2`, and we make sure to include the CFlags for `libxml2`.

2. Look at the first fifteen or so lines (up to the point where `pi` is defined) of the main function in `diffeq_oscillations_xml.cpp` (you can ignore the cryptic `LIBXML_TEST_VERSION` line). *Describe what is happening up to that point.*

→ So we include the libraries and headers we need, and define function prototypes and structures. Then, after the main header, we use the `LIBXML_TEST_VERSION` macro to check that we can use XML. Then we interpret the command line arguments and error output if we don't have two arguments. Then we read our argument XML file and output an error to `cerr` if we can't find the file or if the XML is invalid.

3. Run the program using `./diffeq_oscillations_xml.x oscillation_parameters.xml`. *Did it work?*

→ Yes it works.

4. Make a copy of `oscillation_parameters.xml` and change some of the values in the copy. Then run the program with the new input file again and convince yourself that the parameters really changes.
5. The original version of the program allowed the user to choose if the output for the new run is appended to the output file or if the output file is going to be overwritten. The XML version still has this functionality. Read through the source code of `diffeq_oscillations_xml.cpp` and figure out what you need to change in the XML file to append to the output file. Make that change and try it out. *What did you need to change in the XML file?*

→ You need to add the tag `Append` to the added `Output` tag to get it to append.

E) Reflection

Write down which of the six learning goals on the syllabus this worksheet addressed for you and how:

→ This worksheet helped me learn some more tools of computational physics, like pipes, grep, and XML.